

The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective

Jiin Kim
KAIST
jiin.kim@kaist.ac.kr

Byeongjun Shin
KAIST
byeongjun.shin@kaist.ac.kr

Jinha Chung
KAIST
jinha.chung@kaist.ac.kr

Minsoo Rhu
KAIST
mrhu@kaist.ac.kr

Abstract—Large-language-model (LLM)–based AI agents have recently showcased impressive versatility by employing dynamic reasoning, an adaptive, multi-step process that coordinates with external tools. This shift from static, single-turn inference to agentic, multi-turn workflows broadens task generalization and behavioral flexibility, but it also introduces serious concerns about system-level cost, efficiency, and sustainability. This paper presents the first comprehensive system-level analysis of AI agents, quantifying their resource usage, latency behavior, energy consumption, and datacenter-wide power consumption demands across diverse agent designs and test-time scaling strategies. We further characterize how AI agent design choices, such as few-shot prompting, reflection depth, and parallel reasoning, impact accuracy-cost tradeoffs. Our findings reveal that while agents improve accuracy with increased compute, they suffer from rapidly diminishing returns, widening latency variance, and unsustainable infrastructure costs. Through detailed evaluation of representative agents, we highlight the profound computational demands introduced by AI agent workflows, uncovering a looming sustainability crisis. These results call for a paradigm shift in agent design toward compute-efficient reasoning, balancing performance with deployability under real-world constraints.

Index Terms—AI agents, test-time scaling, energy consumption, infrastructure sustainability.

I. INTRODUCTION

Recent progress in large language models (LLMs) has shifted from scaling model size or pretraining data to improving inference-time behavior, a direction known as *test-time scaling* [48], [75]. Test-time scaling is designed to enhance model performance by allocating additional computation during inference without modifying the model’s parameters. This includes techniques such as Chain-of-Thought [84], Tree-of-Thought [95], and others [7], [63], [79], [83], [104]. These approaches promote more deliberate and interpretable *reasoning* within the LLM, enabling it not only to recognize patterns but also to derive conclusions, generate explanations, and solve tasks that require step-by-step logic.

Deploying these reasoning-enhanced LLMs, however, comes at an immense computational cost. Even in current **static reasoning** models which follow fixed input-output mappings without external tool interaction (Figure 1(a,b)), LLMs run on thousands of GPUs, whose power, cooling, and capital costs drive monthly expenses into the tens of millions of dollars [69]. A single ChatGPT query is estimated to consume about ten times the electricity of a typical web search [12] and requires a substantial amount of cooling water [62].

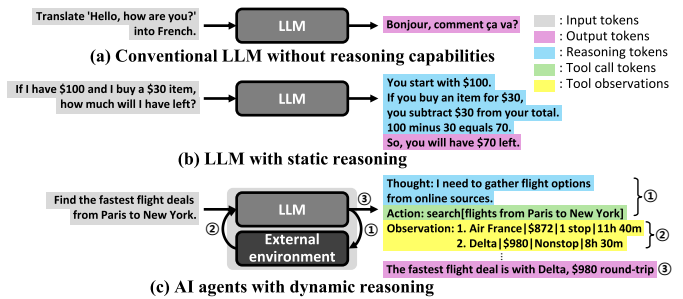


Fig. 1. Overview of reasoning strategies in LLM-based systems. (a) Conventional LLMs map inputs directly to outputs in a single forward pass, with no explicit intermediate reasoning. (b) Reasoning-enhanced LLMs internally create intermediate steps (e.g., sampling alternative responses or extending token sequences) to deepen or diversify their thought process. (c) AI agents augment this reasoning by ① planning and invoking external tools, ② observing the outcomes and adapting their internal reasoning, and iteratively refining their decision-making until they ③ generate the final answer.

As a result, hyperscalers are investing at an unprecedented scale. Meta has already committed over \$10 billion to AI infrastructure [66], and Microsoft operates custom AI supercomputers that draw tens to hundreds of megawatts. As an example, xAI’s Colossus AI supercomputer alone employs approximately 100,000 Nvidia H100 GPUs, consuming 150 megawatts in total, with the GPUs alone accounting for around 70 megawatts [88]. For perspective, traditional hyperscale data centers typically draw between 10 and 100 megawatts [27], while advanced semiconductor fabs, such as those operated by Samsung and TSMC, consume several hundred megawatts each [28], [80]. Analysts forecast that total AI infrastructure spending will surpass \$1 trillion within this decade [14], raising serious concerns about power grid sustainability and the economic viability of large-scale LLM deployment.

Given this landscape, the emergence of *AI agents* powered by LLMs with **dynamic reasoning** threatens to exacerbate these already formidable infrastructure pressures dramatically. Unlike static reasoning models, dynamic reasoning represents an advanced form of test-time scaling that significantly boosts capabilities through active interaction with external environments (Figure 1(c)). Specifically, AI agents continuously plan, invoke external tools, observe outcomes, and iteratively refine their reasoning, often performing dozens of inference calls to satisfy a single user request [72], [96], [102]. Without substantial system-level innovations, per-request computational

costs could increase by orders of magnitude, making large-scale deployment of agents economically and environmentally prohibitive. Industry leaders are already responding to these challenges: OpenAI’s planned Stargate project [55] and Meta’s next generation AI data center Hyperion [74] are each projected to require multiple “gigawatts” of power capacity, with costs reaching hundreds of billions of dollars. Yet, despite these developments, the computer architecture community has largely focused on static LLMs, leaving the infrastructure implications of dynamic reasoning workloads underexplored.

To address this critical gap, this paper presents a rigorous, quantitative evaluation of the computational and infrastructural costs of dynamic reasoning. We systematically characterize resource utilization, latency implications, and energy demands inherent in the iterative execution patterns of AI agents. While the serving characteristics of static reasoning LLMs are well understood within the research community, our work presents the first comprehensive, system-level characterization of agent serving costs across diverse configurations and workloads. Each component of our analysis—including the characterization of agent workflows themselves, serving performance, and the impact of test-time scaling—quantifies a distinct dimension of this cost structure, collectively building a unified understanding of its infrastructure-level implications. Our analysis highlights the critical system-level challenges faced when deploying AI agents and identifies opportunities for optimization through architectural improvements, enhanced inference algorithms, and intelligent resource allocation strategies. To the best of our knowledge, this work is the first to provide a system-level characterization of dynamic reasoning in AI agents, grounded in quantitative analysis of end-to-end infrastructure behavior across diverse agentic workflows¹. A key contribution and objective of our study is to quantitatively assess the AI infrastructure cost of dynamic reasoning deployments, and to inform and caution the research community about the urgent need for sustainable, efficient design principles to bridge the gap between advanced algorithmic capabilities and practical, scalable, and sustainable deployment.

II. BACKGROUND AND MOTIVATION

A. Definition of AI Agents

AI agents are inference-time frameworks that extend the capabilities of LLMs by enabling multi-step reasoning, adaptive decision-making, and interaction with the external environment. Unlike conventional LLM applications that produce a single output from a static prompt, AI agents operate through iterative internal reasoning and external actions at inference time. At each iteration, the agent may generate an intermediate reasoning result, call an external tool (e.g., search engine, calculator, or code interpreter), and incorporate the output into its subsequent decisions. This process allows the agent to retrieve missing information and refine its strategy *dynamically* in response to evolving task demands. While this

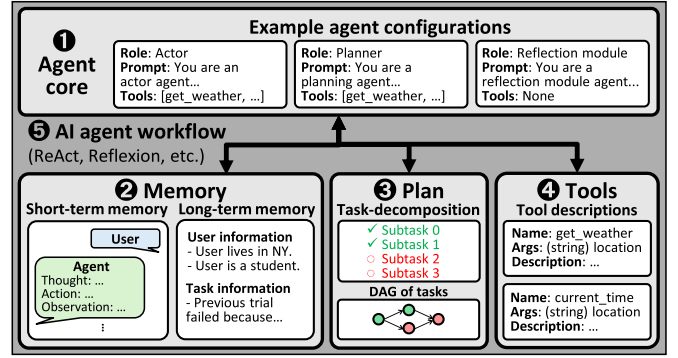


Fig. 2. Overview of AI agent structure.

adaptivity enhances the ability to handle complex and open-ended problems, it also leads to variability in the LLM calls, tool usage patterns, and overall computational cost.

B. Core Components and Workflows of AI Agents

As illustrated in Figure 2, AI agents generally consist of four core components (*agent core*, *memory*, *plan*, and *tools*), and *AI agent workflows* interconnect these core components through iterative interactions. These workflows orchestrate how the components dynamically collaborate, enabling the agent’s adaptive behaviors.

The ❶ *agent core* is the central component responsible for *advanced reasoning*, powered by one or more LLMs configured in specific “roles”. These roles typically include an *actor*, which determines the agent’s next action; a *planner*, which decomposes high-level goals into subtasks; and a *reflection module*, which evaluates prior reasoning steps and tool interaction trajectories to guide future decisions.

This core reasoning capability of the AI agent is further supported by *memory*, *plan*, and *tools*. ❷ *Memory* plays a critical role in enabling the agent to maintain continuity across reasoning steps by storing short-term interaction traces as well as long-term knowledge, including user preferences or experience from past interactions. ❸ *Plan* organizes the agent’s objective into a sequence of subtasks or a directed acyclic graph (DAG) of interdependent actions. By maintaining an explicit plan, the agent can prioritize actions, track progress, and make forward-looking decisions that align with the overall task structure. ❹ *Tools* extend the agent’s capabilities beyond text generation by enabling interaction with external environments. At each step, the agent analyzes its current context, generates a structured command specifying the desired tool and input, executes the tool call, and incorporates the resulting output into its context. This output is then used to guide the next stage of reasoning.

Finally, ❺ *AI agent workflow* defines how an agent leverages interactions among the four core components iteratively to carry out reasoning and coordinate actions. AI agents implement their own distinct workflows, reflecting different coordination patterns among components. These workflows can be broadly decomposed into two phases: (1) *LLM inference phase*, where the agent performs internal reasoning tasks such as action generation, planning, or reflection; and (2) *tool use*

¹Open-sourced at <https://github.com/VIA-Research/AgentBench>.

phase, where the agent interacts with external environments using tools. These two phases alternate iteratively, forming the backbone of AI agentic systems.

C. Test-Time Scaling in AI Agents

Test-time scaling refers to methods that improve the reasoning performance of pretrained LLMs at inference time by increasing the amount of computation used for inference without modifying model parameters [75], [83], [84], [95], [104]. Representative techniques include Chain-of-Thought [84], which guides the model to produce intermediate reasoning steps through carefully crafted prompts, and Tree-of-Thought [95], which expands the reasoning space by exploring multiple reasoning paths. These approaches guide the model to perform step-by-step reasoning, effectively leveraging its internal reasoning capabilities while keeping its parameters fixed.

AI agents build upon this paradigm by implementing test-time reasoning not through prompt design alone, but through multi-step decision-making that integrates tool use and maintenance of intermediate reasoning state. Unlike conventional prompt-based methods that operate within a *static* input-output mapping, agents *dynamically* coordinate multiple model invocations and tool interactions, adapting their behavior based on intermediate outcomes. This form of *dynamic reasoning* enables agents to respond to new information, revise prior decisions, and handle real-time tasks involving external environments. As such, AI agents redefine test-time scaling by moving beyond conventional inference approaches that rely solely on the model's internal reasoning abilities. This paradigm shift introduces new challenges in efficiency, latency, and resource management, highlighting the need for a system-level analysis of the behavior of AI agents.

D. Motivation

Unlike conventional single-turn LLM inference where computation is bounded to a single forward pass, agentic execution involves dynamically evolving control flows, multiple rounds of LLM inference, and external tool interactions. These behaviors introduce profound challenges at the systems and infrastructure level, incurring significant compute overhead, amplifying memory pressure, and introducing unpredictable latency and resource utilization patterns.

Despite these operational complexities, prior research on AI agents has largely focused on improving task success rates and qualitative reasoning behavior [72], [96], [102], with little attention paid to its deployment costs. Questions central to the deployment and scaling of such agents remain largely unexamined. Consequently, existing architecture and systems optimizations for LLMs, which target static, single-pass workloads, may fall short in capturing or addressing the dynamic and iterative characteristics unique to AI agents.

This paper is motivated by the urgent need to fill this gap. To the best of our knowledge, this work is the first to present a rigorous, system-level characterization of AI agents, grounded in quantitative measurement across diverse agent designs and tasks. We argue that without a principled understanding of

TABLE I
COMPARISON OF AI AGENTS.

Agent	Reasoning	Tool Use	Reflection	Tree Search	Structured Planning
CoT [84]	O	X	X	X	X
ReAct [96]	O	O	X	X	X
Reflexion [72]	O	O	O	X	X
LATS [102]	O	O	O	O	X
LLMCompiler [31]	O	O	O	X	O

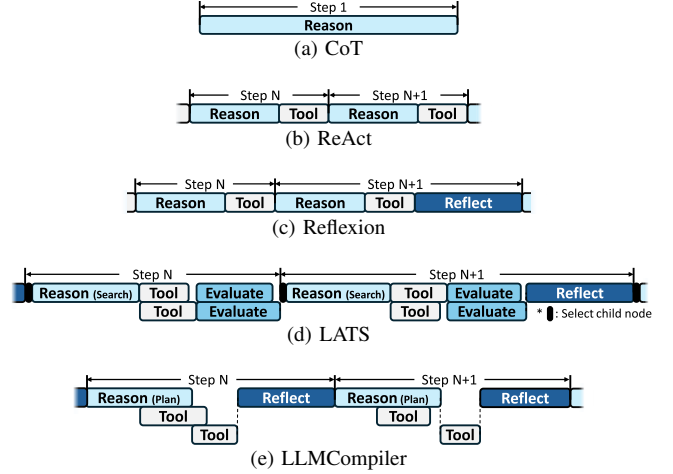


Fig. 3. Execution timeline of each AI agent.

the system-level implications of dynamic reasoning, the community risks building infrastructure optimized for yesterday's workloads. A systems-oriented perspective is therefore critical to guide the design of sustainable, efficient, and scalable serving infrastructures. Our study takes this first step by analyzing the computational and infrastructural costs of deploying AI agents in practice, providing actionable insights for future architecture and systems co-design.

III. METHODOLOGY

Our analysis considers a representative set of AI agents and benchmarking workloads, covering diverse agent workflows and agentic task characteristics.

AI agent workflows. We investigate five representative agents: Chain-of-Thought (CoT) [84], ReAct [96], Reflexion [72], Language Agent Tree Search (LATS) [102], and LLMCompiler [31]. These agents were selected to cover a wide range of reasoning strategies, tool integrations, and planning mechanisms. Table I summarizes the presence or absence of five key capabilities across each agent.

- **Reasoning.** All agents considered in this study employ a reasoning mechanism. Among them, CoT operates purely through internal reasoning without the use of any external tool (Figure 3(a)). As a baseline for comparison, CoT-style static reasoning approaches are considered within the broader definition of AI agents, despite their lack of external interactions with tools.
- **Tool use.** Tool use differentiates purely language-based agents from those capable of interacting with the external

TABLE II
DESCRIPTION OF BENCHMARKS.

Benchmark	Property	Description
HotpotQA [92]	Task	Multi-hop question answering
	Tool	Wikipedia APIs (search, lookup keywords)
	Agent	CoT, ReAct, Reflexion, LATS, LLMCompiler
WebShop [94]	Task	Online shopping
	Tool	Interactive web navigation (search, click)
	Agent	ReAct, Reflexion, LATS, LLMCompiler
MATH [25]	Task	Math problem solving
	Tool	Wolfram Alpha API, Python-based calculator
	Agent	CoT, ReAct, Reflexion, LATS
HumanEval [10]	Task	Programming
	Tool	Executing self-generated test code
	Agent	CoT, ReAct, Reflexion, LATS

environment. This functionality enables agents to access real-time data or perform non-linguistic operations.

- **Reflection.** Reflection allows agents to evaluate past decisions and revise strategies accordingly. Reflective agents effectively manage *long-term memory* by abstracting past trajectories into reflections. While ReAct agents simply repeat reasoning and tool usage (Figure 3(b)), Reflexion, the most fundamental reflective agent, enhances adaptability by periodically incorporating self-evaluation and refinement through reflection (Figure 3(c)).
- **Tree search.** LATS (Figure 3(d)) leverages Monte Carlo Tree Search [11] to simulate multiple branches of reasoning and action, allowing the agent to evaluate different candidate paths before making a decision. By simulating multiple possible future paths, the agent can make more informed decisions and select optimal action sequences.
- **Structured planning.** LLMCompiler incorporates a structured multi-step planning and streaming for asynchronous task execution to minimize latency. During the planning phase, LLMCompiler analyzes task dependencies and constructs a DAG that organizes future tool calls into an execution plan. This enables multiple dependent tool calls to be generated within a single LLM invocation. As the plan is constructed, intermediate tool calls are streamed to the execution stage, allowing the scheduler to overlap planning and tool calls via asynchronous execution. Together, these features can help reduce repeated reasoning and lower end-to-end latency (Figure 3(e)).

In general, we utilized the official open-source implementations provided by the original authors of these agent workflows [30], [71], [97], [103]. Each AI agent is adapted to support our evaluation framework and benchmarks. For LATS, we further optimized its implementation to support concurrent LLM inference and parallel tool invocation because the original version [103] executes these operations sequentially, aggravating end-to-end latency.

Benchmarks. We select four popular benchmarks representative of various downstream agentic tasks, whose descriptions are summarized in Table II. HotpotQA [92] is a question-answering benchmark that assesses the agent’s ability to accurately retrieve relevant evidence to answer multi-hop knowledge-intensive questions. We provide the Wikipedia

APIs [85] as tools to solve these questions. WebShop [94] is a web-shopping benchmark where agents find the best-fit item that meets the given conditions. The agent is given web navigation tools to browse WebShop. MATH [25] is a benchmark suite of mathematics problems across various domains. Agents are equipped with access to the Wolfram Alpha API [86] for solving complex equations, as well as a Python-based calculator for simple numerical computations. HumanEval [10] evaluates the programming capability of agents. In our setup, agents are equipped with a Python execution tool that allows them to validate the generated solutions by executing self-written test code. In addition to these agentic benchmarks, we utilize a *non-agentic* dataset, which is the ShareGPT dataset [70], to model conventional chatbot-like LLMs, characterized by single-turn LLM inference without iterative interactions with the external environments. ShareGPT contains a collection of real conversations between users and ChatGPT [53], capturing standard interactive dialogue scenarios.

It is worth pointing out that some “AI agent vs. benchmark” pairs are omitted if the agent is not suitable for solving the target task. For example, CoT is excluded from WebShop since it cannot interact with the shopping webpage. Similarly, LLMCompiler is omitted from MATH and HumanEval, as its DAG-style planning is not well-suited for problems that require sequential, step-by-step reasoning and tool usage.

LLM backend. We employed the OpenAI-compatible vLLM (version 0.6.6) server as the LLM serving infrastructure, integrated with PyTorch 2.6 and CUDA 12.8. We enabled *prefix caching* [32], which reduces redundant computation by reusing previously computed attention states (i.e., *Key-Value cache (KV cache)*) for shared input prefixes across LLM requests. Unless explicitly stated otherwise, all experimental results are obtained with prefix caching enabled. We use Llama-3.1-8B-Instruct [45] as the default backend LLM. However, to discuss the impact of model size on cost and accuracy, we also use Llama-3.1-70B-Instruct [44] in Section V.

Hardware. Experiments were conducted on Google Cloud Platform (GCP). For the 8B model, we used the a2-highgpu-1g instance type with 12 vCPUs (6 physical cores), 85GB memory, and a single NVIDIA A100 40GB GPU². For the 70B model, we used the a2-highgpu-8g instance type with 96 vCPUs (48 physical cores), 680GB memory, and 8 NVIDIA A100 40GB GPUs.

IV. DEMYSTIFYING AI AGENTS

Section IV-A first examines an agent’s *single-request* execution, followed by a detailed exploration of the LLM

²In this work, we use GPU-based serving systems for our analysis because they are the de facto standard for large-scale LLM serving. Our characterization methodology and findings are architecture-agnostic and directly transferable to other accelerator platforms like Google TPUs. As detailed in the rest of this paper, key insights such as the impact of agentic control-flow serialization, long-context KV cache pressure, and idle-period underutilization are inherent to the workload characteristics of dynamic reasoning, not to any GPU/TPU-specific microarchitecture. Thus, the system-level implications we identify remain equally relevant to other AI inference accelerators, providing a foundation for future cross-architecture analyses.

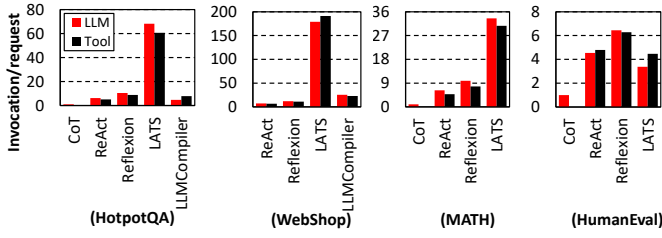


Fig. 4. Average number of LLM and tool invocations per request.

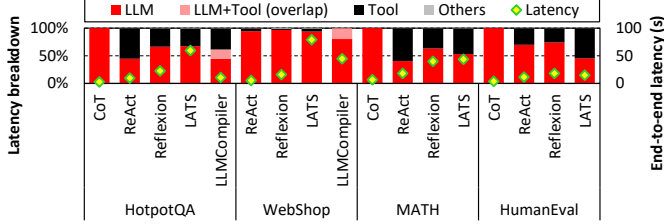


Fig. 5. Latency breakdown of agents (left axis, bar graph) and their end-to-end latency for processing a single request (right axis, diamond marker). The pink bars represent phases where LLM and tool execution latencies overlap, as observed in LLMCompiler, which asynchronously executes tools during plan generation.

inference and tool-calling characteristics of agents in Section IV-B. Lastly, Section IV-C shifts the focus to the serving environment of agentic systems where *multiple* requests are handled concurrently, identifying system-level bottlenecks and scalability issues that emerge in agent deployment.

A. Overall Workflow of AI Agents

Effect of LLM and tool calls on latency. Figure 4 shows the average number of LLM and tool invocations per request across benchmarks. While CoT performs only a single LLM inference per request, tool-augmented agentic systems require significantly more LLM calls, averaging 9.2 times more than CoT. Among these, LATS exhibits the highest LLM invocation count, with an average of 71.0 LLM calls per request. This is primarily due to its use of tree search, which explores multiple reasoning branches (i.e., child nodes) by issuing separate LLM inferences for each one when expanding a tree node.

Figure 5 presents the end-to-end latency and the latency breakdown of each agent’s execution. While most agents exhibit a similar number of LLM and tool calls per request (Figure 4), the latency contribution from tool calls varies significantly depending on the workload. This discrepancy is primarily due to differences in the underlying tool execution latencies. For example, WebShop uses lightweight tools that interact with locally hosted webpages, resulting in tool latencies as low as 20 ms per call. In contrast, HotpotQA relies on the Wikipedia API, where individual calls take an average of 1.2 seconds. As a result, tool execution dominates the overall latency breakdown in this case.

On average, LLM inference and tool execution account for 69.4% and 30.2% of total latency, respectively. Both stages contribute significantly to overall latency, but they are difficult to overlap due to their sequential dependency. Specifically, the LLM output is needed to determine which tool to invoke

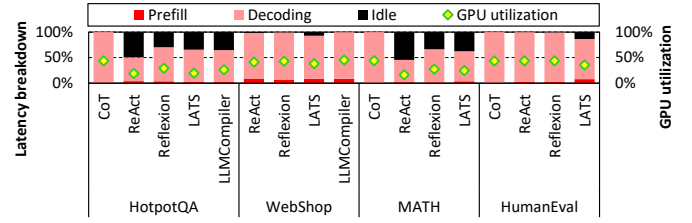


Fig. 6. Breakdown of GPU runtime by usage (left axis, bar graph) and the resulting average GPU utilization (right axis, diamond marker). GPU utilization is measured as the fraction of actively used GPU cores, using NVIDIA’s DCGM [51].

along with the required arguments. Conversely, the next LLM invocation typically relies on the observation returned by the tool. Although LLMCompiler attempts to mitigate this dependency by streaming intermediate plans to the scheduler for asynchronous execution of tool calls (thus concurrently executing it with planning), the observed overlap accounts for only 18.2% of total latency.

Agentic workflow’s effect on GPU compute utility. Figure 6 breaks down the GPU runtime by usage and reports the resulting average GPU utilization when handling a single request. Although this setup assumes the processing of a single agent task, concurrent LLM calls can be opportunistically batched to improve GPU utilization, whenever possible, to more efficiently execute agents such as LATS. Unlike CoT, which performs a single LLM inference without external interaction, it is possible for agents to experience longer GPU idle periods due to tool execution. The duration of these idle periods depends on the tool’s latency and whether it leverages the GPU. In WebShop, the tool interacts with locally hosted synthetic web pages, resulting in very short tool latencies (Figure 5), so agents do not experience notably higher GPU idle time (i.e., lower GPU utilization). HumanEval exhibits longer tool execution times (Figure 5), but the proportion of GPU idle time remains minimal because the tool it calls (which is the test generation tool) utilizes the GPU for LLM execution. In contrast, HotpotQA and MATH employ tools that operate on local CPUs or external systems, leading to substantial GPU idle periods that account for up to 54.5% of total execution time, resulting in significantly lower GPU utilization compared to CoT. When the GPU is executing the LLM, its activity can be further divided into the prefill and decode stages, which account for 4.7% and 74.1% of the GPU’s execution time, respectively. As noted in [2], [4], [8], [34], the decode stage is known to be memory-bound. Consequently, the large fraction of time spent in the decode stage further contributes to the underutilization of GPU resources.

Because the sequential dependency between LLM inference and tool calls limits parallel execution opportunities within a single request (i.e., intra-request parallelism), improving overall resource utilization requires leveraging *inter-request* parallelism. We explore this direction in Section IV-C, where we discuss the implications of serving AI agents over multiple queries with LLM request batching [32], [82], [98].

End-to-end latency distribution of AI agents. Figure 7

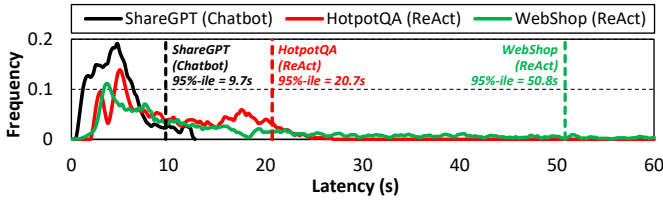


Fig. 7. End-to-end latency distribution of a non-agentic ShareGPT workload and ReAct-based agents with HotpotQA and WebShop workloads. Latency is measured while processing one request at a time, with prefix caching enabled.

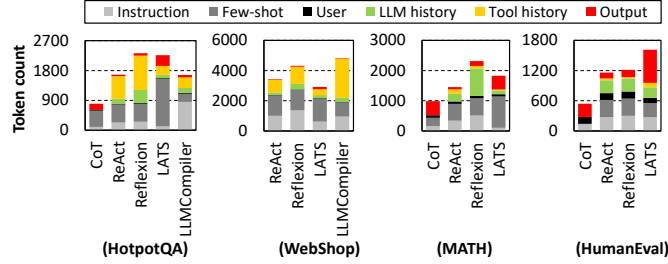


Fig. 8. Breakdown of input and output tokens in LLM inference. *Instruction* and *Few-shot* (light and dark gray) represent input tokens that are statically fixed as part of the initial prompt to the LLM. *User* (black) denotes input tokens provided by the user as part of the query. *LLM history* (green) and *Tool history* (yellow) represent tokens accumulated from previous LLM outputs and tool responses, respectively, which are then included as input tokens during the next LLM call. *Output* (red) refers to tokens generated by each LLM call.

compares the end-to-end latency distributions of a conventional, non-agentic LLM service using ShareGPT and a ReAct-based agent system using HotpotQA and WebShop. The ShareGPT dataset represents a typical chatbot workload, where each response is generated by a single LLM inference. As shown, this results in a relatively low and consistent latency distribution, with most responses completing within 9.7 seconds. In contrast, the ReAct-based agent exhibits a much broader latency distribution with a heavier tail, due to its multi-step reasoning and reliance on external tools. Because the number of reasoning steps and tool calls varies across requests in agents, the associated computational demands also fluctuate. Consequently, there is significant variance in latency across queries targeting agents.

B. LLM Inference and Tool-Calling Characteristics

This section further analyzes the behavior of agentic systems by characterizing the properties of LLM inference and tool calls within the AI agent in greater detail.

Breakdown of input and output tokens in LLM inference. Figure 8 presents the token count distribution across different AI agents. *Instruction* tokens define the agent’s role and objective within the task, while *Few-shot* tokens provide in-context examples that guide the agent’s behavior. *User* tokens represent user queries. *LLM history* and *Tool history* tokens consist of accumulated outputs from previous LLM inferences and tool responses across iterations. *Output* tokens are generated at each LLM inference step, while the remaining tokens collectively make up the input prompt.

Compared to CoT, AI agents generally have longer input tokens. This is because their inputs include additional elements

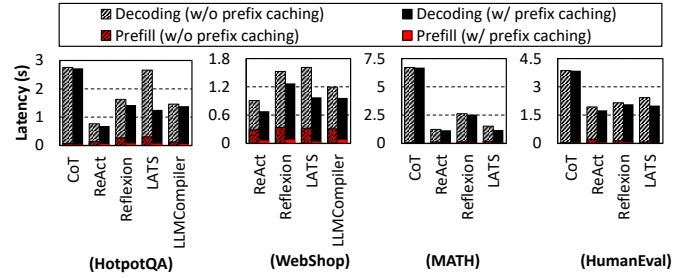


Fig. 9. Inference latency with(out) prefix caching and its breakdown.

such as agent role-aligned instructions (e.g., LLMCompiler requires instructions to generate a structured plan) and the accumulated context of previous LLM and tool interactions. For output tokens, each LLM call in agent workflows often generates fewer tokens than CoT, except for LATS. This is because agents typically decompose a single task into multiple steps, distributing the overall output across several LLM calls. In contrast, LATS often generates much longer outputs than CoT due to its workflow, where a single LLM call produces multiple candidate samples to expand the tree node.

Token usage patterns also vary depending on the task workload. In knowledge-intensive tasks such as HotpotQA and decision-making tasks like WebShop, tool calls often return large responses (e.g., the full content of a webpage) resulting in longer tool history tokens. In contrast, tasks that rely more heavily on internal reasoning, such as MATH and HumanEval, tend to produce longer LLM-generated outputs, leading to larger LLM history tokens.

Although the ratio of LLM and tool history tokens varies across workloads, most benchmarks exhibit substantial growth in input history over multiple iterations. An exception is LATS, which includes only the path from the root to the current node, rather than concatenating all prior interaction histories. In the case of HotpotQA, for instance, initial inputs are typically around 1,000 tokens, but the input size increases to 3–4 $\times$ as prior LLM outputs and tool responses are appended to the input context of subsequent LLM calls. Because histories accumulate sequentially, consecutive LLM calls share common prefixes in their input contexts. These long input contexts result in high KV cache usage per request and considerable prefix overlap across iterations. This behavior presents an opportunity to improve GPU compute and memory efficiency through *prefix caching* [32], as detailed below.

Effect of prefix caching on AI agent’s compute efficiency. Building on the token-level analysis above, we now turn to system-level characteristics, starting with GPU compute efficiency. AI agent workloads involve multiple iterative LLM calls, where a large portion of the input context is reused at each step. Prefix caching leverages this shared prefix to skip redundant computation during the prefill phase by reusing previously cached key-value (KV) pairs.

Figure 9 shows LLM inference latency and its proportion of prefill and decoding latency, with and without prefix caching. For CoT, LLM inference occurs only once per request, and the shared prefix across inferences is minimal. Moreover, CoT

typically generates a relatively large number of output tokens, making decoding the dominant contributor to latency. In contrast, AI agents operate iteratively and accumulate long input contexts due to interaction histories. As a result, prefix caching reduces prefill latency by an average of 60.1%, demonstrating its effectiveness in improving compute efficiency by avoiding redundant computations through prefix reuse. Beyond the pre-fill phase, prefix caching can also indirectly improve decoding efficiency. In systems that execute multiple parallel LLM calls, decoding may be stalled by ongoing prefill operations. By reducing prefill latency, prefix caching shortens this blocking period, thereby enabling faster decoding and explaining the larger speedup observed in LATS.

Overall, the impact of prefix caching on end-to-end LLM inference latency varies by workload type. While CoT workloads benefit less due to their decoding-dominant property, agentic workloads experience an average 15.7% reduction in end-to-end latency due to the accumulation of long input contexts over iterative steps. While this per-request improvement may seem modest, the reduction in prefill time can significantly alleviate system-level bottlenecks. In token-level schedulers like vLLM, long prefill phases can delay the scheduling of concurrent requests. By shortening these phases, prefix caching can improve scheduling efficiency and increase overall system throughput. This effect is examined further in Section IV-C (Figure 11).

Effect of prefix caching on AI agent’s memory efficiency.

We now discuss the effect of prefix caching on GPU memory requirements by measuring the average GPU memory required to store the KV cache. On average, tool-augmented AI agents consume $3.0\times$ more memory per request than CoT, and up to $5.4\times$ more in the worst case. This overhead arises from the iterative nature of agent workflows, where each LLM call appends intermediate reasoning steps and tool responses to the context, resulting in a longer input for each LLM inference.

These results highlight the need for memory optimization in AI agent workloads, with prefix caching serving as a key technique for reducing GPU memory usage. In LATS, multiple LLM inferences are issued in parallel to evaluate several child nodes simultaneously during tree expansion. Without prefix caching, each of these parallel calls creates its own KV cache, resulting in significant memory overhead due to redundancy. With prefix caching, the shared prefix across these parallel calls can be reused, reducing memory requirements by an average of 64.8% in LATS. For other agents, where all LLM calls are invoked sequentially, prefix caching does not reduce memory usage *within a single request*, since the KV cache cannot be shared across LLM calls. However, in serving scenarios with concurrent requests, prefix caching can significantly improve memory efficiency by reusing the KV cache across requests. We further explore this serving-level memory efficiency in Section IV-C (Figure 12).

C. AI Agent Serving Characteristics

So far, our characterization has focused on the behavior of AI agents when servicing a single query for a specific task. In this section, we shift our attention to system-level properties

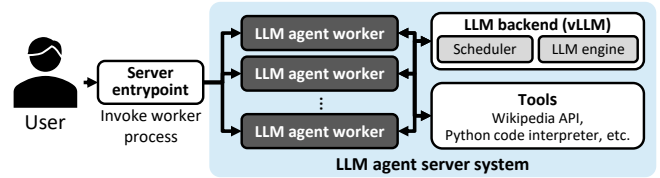


Fig. 10. High-level overview of our AI agent serving system.

of AI agent serving environments, analyzing scenarios where multiple requests are routed to the server and can be processed concurrently for high serving throughput. Unlike static reasoning models that process a user request with a single LLM inference step, AI agents perform multiple reasoning steps iteratively, introducing new challenges for efficient serving.

To examine the characteristics of AI agent serving, we implement an agent serving system, as illustrated in Figure 10. When a user sends a request to the agent server’s entry point, each worker processes the request according to the agent’s workflow. Depending on the current step of the task, a worker either sends a request to the LLM inference server or executes a tool. Tool execution may occur locally (e.g., code interpreters, custom functions) or involve external resources (e.g., web search, API calls). Each worker operates asynchronously, and LLM inference requests from multiple workers can be batched at the LLM backend (e.g., vLLM) for high-throughput processing using continuous batching [32], [98]. We adopt vLLM’s default first-come-first-served (FCFS) scheduler in the LLM inference backend. To simulate realistic traffic, input queries to the agent server are randomly sampled and issued to the server following a Poisson arrival distribution [47].

Importance of concurrent request scheduling. Before comparing the AI agent serving against the conventional chatbot (ShareGPT) serving, we first highlight the importance of concurrently servicing AI agent requests. When ReAct agents are executed *sequentially*, the average latency is 9.6 seconds for HotpotQA and 5.3 seconds for WebShop, limiting throughput to 0.10 and 0.19 queries per second (QPS), respectively. With *concurrent execution*, throughput improves to 2.6 and 1.2 QPS for HotpotQA and WebShop (Figure 11), respectively, achieving $25\times$ and $6.2\times$ gains at the cost of a $2.1\times$ increase in average latency. The greater throughput gain in HotpotQA comes from its longer tool latency, which causes the GPU to remain idle for extended periods. These idle intervals can be effectively utilized by servicing other requests, enabling higher concurrency and throughput.

Comparison with conventional static reasoning LLM services. We now compare an AI agent serving with a conventional LLM serving scenario, represented by the chatbot (ShareGPT) workload. ShareGPT, a typical single-turn LLM service, processes user queries in a single inference pass. Figure 11 shows the changes in end-to-end tail latencies for chatbot (ShareGPT) and ReAct-based AI agent (HotpotQA and WebShop) workloads as input QPS to the server increases. The peak throughput is measured as the maximum sustainable QPS at the knee of the tail latency curve. As depicted, the

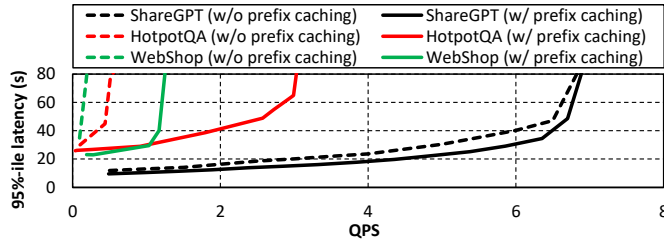


Fig. 11. 95th percentile latency for chatbot (ShareGPT) and ReAct-based AI agents (HotpotQA and WebShop) as QPS rates increase, with (solid line) and without (dashed line) prefix caching enabled.

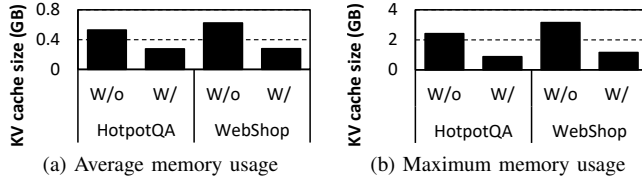


Fig. 12. (a) Average and (b) maximum memory used for KV caches, with and without prefix caching. Evaluation is conducted at 0.2 QPS (HotpotQA) and 0.1 QPS (WebShop) data points using ReAct.

peak throughput of ReAct is significantly lower than that of ShareGPT. While ShareGPT can sustain up to 6.4 QPS, ReAct supports only 2.6 QPS on HotpotQA and 1.2 QPS on WebShop, even with prefix caching enabled. This limitation stems from ReAct’s multi-step reasoning, where each request involves multiple LLM calls and tool interactions, significantly increasing latency.

Effect of prefix caching on AI agent serving throughput.

Prefix caching is an important system-level optimization that reduces redundant computation during the prefill phase of LLM inference by reusing previously computed key-value (KV) caches. While its impact on the latency of individual LLM calls is modest (Figure 9), it can substantially improve throughput and serving efficiency for AI agents.

Figure 11 compares the effect of prefix caching on chatbot (ShareGPT) and agentic (ReAct) workloads. ShareGPT shows only a modest $1.03\times$ throughput improvement, as it performs a single LLM call per request with minimal repetition. In contrast, ReAct benefits significantly, achieving an average $5.62\times$ increase in throughput. This is because agent workloads involve multiple LLM calls per request, amplifying the benefits of avoiding redundant prefill operations.

The performance gap is further explained by token-level batching systems such as vLLM. Without prefix caching, long prefill stages occupy the GPU and block decoding for other requests, leading to system-wide queuing delays. This bottleneck is particularly problematic for AI agents, where repeated LLM calls per request exacerbate inter-request contention. As a result, prefix caching plays a critical role in mitigating these interference effects and improving overall serving efficiency, especially for agentic workloads.

Effect of prefix caching on AI agent’s memory usage.

We now investigate the impact of prefix caching on GPU memory efficiency in AI agent serving, focusing specifically on its effect on key-value (KV) cache size, one of the most

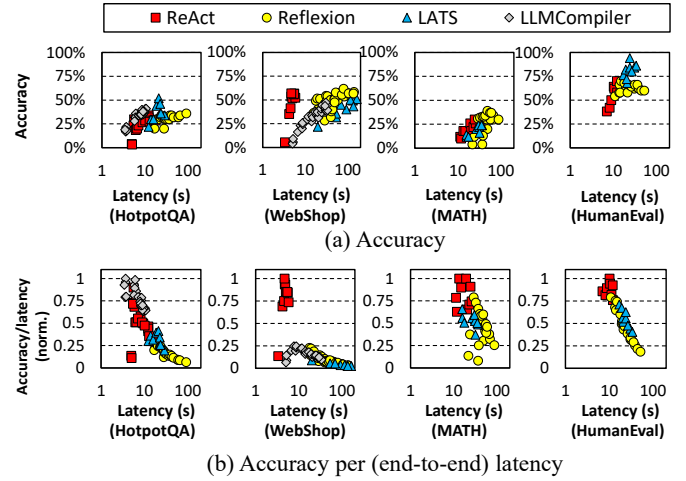


Fig. 13. Accuracy and cost-efficiency of AI agent design points. (a) Accuracy vs. latency and (b) Accuracy per latency, illustrating how efficiently each configuration translates cost into task performance.

significant contributors to memory usage in LLM inference. Figure 12 shows the GPU memory consumption for KV cache allocation, with and without prefix caching enabled, under identical QPS conditions. With prefix caching enabled, the average and maximum KV cache memory usage decrease by 51.7% and 63.5%, respectively, indicating improved memory efficiency. This reduction arises from the ability of prefix caching to reuse key-value pairs of shared prefix tokens across multiple LLM invocations across AI agent requests. Thus, prefix caching not only improves compute efficiency by eliminating redundant prefill operations but also reduces the KV cache memory footprint, enabling more efficient utilization of GPU memory during AI agent serving.

V. DEMYSTIFYING TEST-TIME SCALING IN AI AGENTS

We now explore the diverse design space of AI agents and examine their test-time scaling behavior to understand the trade-offs between model accuracy and cost. We evaluated accuracy following the official evaluation protocol of each benchmark. For HotpotQA and MATH, we report exact match accuracy, allowing minor formatting variations (e.g., equivalent mathematical expressions) in MATH. For WebShop, we use the task-specific score defined in the benchmark. For HumanEval, accuracy denotes the proportion of tasks that successfully pass all unit tests. To assess each design point, we used a benchmark of 50 sample questions and measured the average accuracy and the computation cost for each.

A. Analyzing Cost-Efficiency Across AI Agent Design Spaces

Deploying AI agents in practical settings requires careful configuration of agentic system parameters. These design choices significantly affect not only the agent’s task success rate but also the overall cost of operating such systems. In this section, we quantify how different parameter configurations in AI agents influence both accuracy and cost-efficiency.

Pareto analysis of accuracy and cost across AI agent designs. Figure 13 presents the trade-off between accuracy

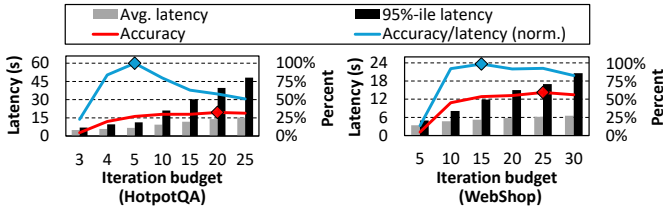


Fig. 14. End-to-end latency and accuracy trends under iteration budget constraints in ReAct. Markers indicate the points of maximum accuracy (red diamond) and peak cost-efficiency (blue diamond), as measured by accuracy-to-latency ratio.

and cost across various AI agent configurations. Each point corresponds to a specific design variant, such as changes to the number of few-shot examples or maximum iteration limits.

Figure 13(a) shows the trade-off between accuracy and latency. ReAct demonstrates strong compute efficiency across all benchmarks, achieving moderate accuracy with consistently low latency. Reflexion builds on ReAct by introducing reflection steps guided by internal or external rewards. This approach yields modest accuracy improvements but significantly increases latency. LATS extends Reflexion with a tree-based reasoning approach that explores multiple candidate branches at each step. While this leads to higher accuracy, it also introduces substantial computational overhead due to the expansion of reasoning paths. LLMCompiler, with its planning-based architecture, outperforms ReAct on tasks like HotpotQA in both accuracy and cost-efficiency, thanks to its ability to generate and execute structured plans in parallel. However, in tasks such as WebShop—where tool usage involves high interdependencies (e.g., searching or clicking on a webpage)—its DAG-style planning results in unnecessary tool invocations, leading to lower efficiency than ReAct.

Figure 13(b) illustrates the cost-efficiency of various agent configurations. We define cost-efficiency as the ratio of accuracy to cost, where cost is measured as end-to-end latency. This metric reflects how effectively each configuration translates compute resources into task accuracy³. Across all agents and workloads, we observe a consistent pattern: *as computation cost increases, accuracy improves, but with diminishing returns*. This underscores the importance of designing AI agent serving systems that find configurations on (or near) the Pareto frontier, optimally balancing model accuracy against deployment cost rather than optimizing solely for accuracy.

Tuning Iteration and Prompting for Cost-Efficient Agent Behavior. To better understand the accuracy–cost trade-offs in AI agent design, we analyze how two key parameters in AI agent designs affect model performance: the maximum iteration budget and the number of few-shot examples.

Figure 14 shows how varying the iteration budget impacts average latency, 95th percentile latency, and accuracy. The iteration budget controls how many reasoning steps and tool invocations the agent is allowed per query. As this budget increases, agents can perform deeper reasoning, which ini-

³Using FLOPs as a proxy for cost (“accuracy per FLOP”) yielded similar qualitative conclusions, so we omit those results for brevity.

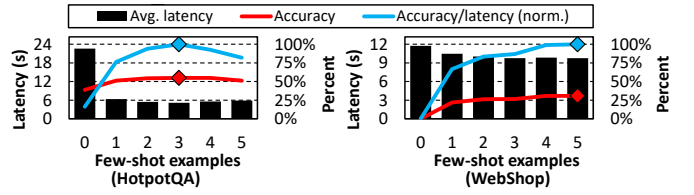


Fig. 15. End-to-end latency and accuracy trends with varying numbers of few-shot examples in ReAct. Markers indicate the configuration with the highest accuracy (red diamond) and the peak cost-efficiency (blue diamond), based on normalized accuracy-to-latency ratio.

tially improves accuracy. However, both accuracy and average latency eventually saturate, while the 95th percentile latency continues to increase linearly. This rising tail latency is driven by a small set of outlier tasks that consume the full iteration budget. These outliers degrade cost-efficiency by contributing disproportionately to total compute usage without yielding substantial accuracy gains. The widening latency distribution also reduces predictability, which is especially problematic for latency-sensitive deployments. Therefore, iteration limits should be tuned not only for performance but also for latency consistency and operational stability.

Figure 15 shows how varying the number of few-shot examples in the prompt affects latency and accuracy. Initially, adding examples substantially improves accuracy, as agents gain better task understanding. However, beyond a certain point, the benefit diminishes—and in some cases, accuracy declines due to prompt length exceeding the model’s optimal processing range. Interestingly, average latency decreases as more examples are added. This counterintuitive result arises because good examples help agents solve tasks in fewer steps, offsetting the cost of longer prompts. Thus, while longer prompts marginally increase per-token processing time, the reduction in overall reasoning steps often leads to net latency savings. In summary, a small number of carefully chosen examples can improve both accuracy and efficiency, while excessive prompting may lead to diminishing returns.

To identify optimal configurations, we highlight the point at which the accuracy-to-latency ratio is maximized (denoted by blue markers in Figure 14 and Figure 15). This point represents the most cost-effective trade-off between model accuracy and response time. Such metrics provide a practical guideline for setting iteration budgets and few-shot prompting under latency or compute constraints.

B. Test-Time Scaling of AI Agents

AI agents can dynamically scale their reasoning at test time by adjusting the number of reasoning steps based on task difficulty. This flexibility helps improve performance on complex problems, but it also introduces significant variation in computation cost. Designing systems that are both accurate and efficient requires a deeper understanding of how inference behavior evolves as compute usage increases.

Sequential vs. parallel reasoning at test time. We investigate the effect of two key forms of test-time scaling for AI agents: *sequential* and *parallel*. In *sequential scaling*,

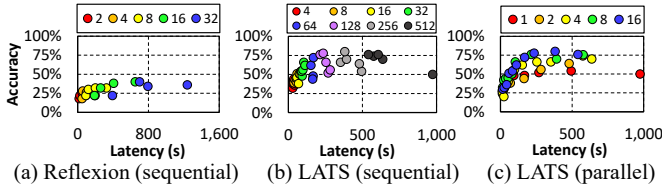


Fig. 16. Accuracy-latency trade-offs with sequential, parallel scaling on HotpotQA. Legends denote the scaling level: maximum reflection steps in (a), (b), and number of child nodes per expansion in (c).

the agent gradually increases its reasoning steps over time, allowing for deeper introspection. This is typical of agents like Reflexion and LATS, where the number of reflection steps can be adjusted dynamically. In contrast, *parallel scaling* issues multiple reasoning branches simultaneously, commonly through parallel LLM calls, to explore diverse solution paths. LATS uses this approach by spawning multiple child nodes during each tree expansion step.

Figure 16(a) and (b) show the accuracy-latency trade-offs for Reflexion and LATS under sequential scaling. Both methods improve in accuracy with more reflection steps, but with diminishing returns. For example, in Reflexion, increasing latency from 16.9s to 25.6s yields a 4% accuracy gain. However, achieving the same model accuracy improvement from a later point (56.0s) requires a much larger increase in latency (269.5s), a $31\times$ higher cost for the same marginal gain.

On the other hand, parallel scaling exhibits a different trade-off. Figure 16(c) highlights the behavior under parallel scaling in LATS. Increasing the number of child nodes from 1 to 16 improves accuracy by 14.4 percentage points while simultaneously *reducing* latency by 196.3s on average. This is because evaluating multiple reasoning paths in parallel helps the agent converge on high-quality answers more quickly. However, this comes at the cost of issuing more concurrent LLM requests, which increases memory pressure and may limit scalability in multi-tenant or resource-constrained environments.

These results suggest that AI agent configurations should align with system constraints such as latency budgets and available compute resources. Parallel scaling is effective for latency-sensitive workloads, as it allows the agent to explore multiple reasoning paths at once and reach better answers faster. However, it increases resource usage due to the large number of concurrent LLM calls. In contrast, sequential scaling is better suited for resource-constrained environments. This approach avoids concurrent LLM calls, lowering peak resource demand, but incurs higher latency from step-by-step reasoning.

Model size effects on test-time scaling. We further analyze how model size affects the accuracy-cost trade-offs under different test-time scaling strategies.

Figure 17(a) shows that both the 8B and 70B Llama-3.1-Instruct [44], [45] models eventually reach saturation in accuracy, but they differ in how quickly they reach this point. The 70B model achieves high accuracy with relatively low latency, whereas the 8B model requires much longer inference times to reach similar performance. This trend is echoed in Figure 17(b), which plots total token usage. The 8B model

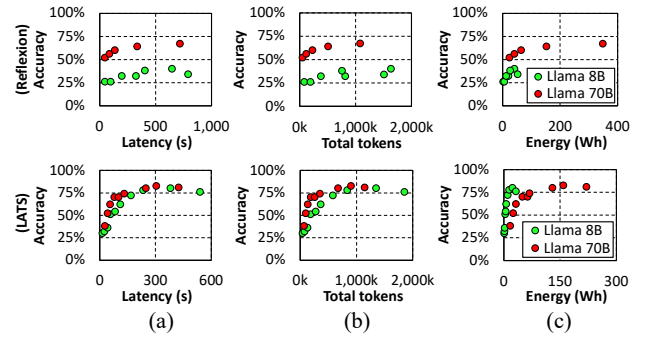


Fig. 17. Accuracy-cost trade-offs under test-time scaling across two model sizes (Llama-3.1-Instruct 8B and 70B) on HotpotQA. (a)–(c) compare Reflexion (top row) and LATS (bottom row) across latency, token usage, and energy consumption. While 70B achieves higher accuracy with fewer steps, the 8B model when paired with parallel scaling can approach 70B performance with lower energy cost. Each point denotes a different level of test-time reasoning.

consumes significantly more tokens at high-accuracy settings, indicating that it needs more reasoning iterations to match the 70B model’s performance. However, as shown in Figure 17(c), the 8B model is substantially more energy-efficient. While the 70B model relies on 8 A100 GPUs, the 8B model runs on just one, resulting in lower total energy consumption per request, even when requiring more reasoning steps to be involved.

Interestingly, the performance gap between models can be partially closed with effective scaling strategies. Reflexion (which uses sequential scaling) shows limited accuracy on the 8B model. But with LATS and parallel scaling, the 8B model achieves near-70B performance by exploring multiple paths and selecting the best one. This shows that a test-time strategy can play a compensatory role in low-resource settings.

VI. AI INFRASTRUCTURE IMPLICATIONS

In this section, we analyze the system-level impact of agentic test-time scaling by quantifying the GPU energy consumption and datacenter-wide power demands of AI agents relative to conventional single-turn LLM inference. Following the methodology in Section V-B, this section utilizes Reflexion and LATS as representative AI agents that employ sequential and parallel scaling, respectively. Reflexion and LATS design points were selected based on the highest-accuracy configurations in Figure 17. Llama-3.1-Instruct 8B and 70B models are used as backend LLMs and ShareGPT serves as the baseline for conventional single-turn inference.

GPU energy consumption. Reflexion consumes 41.53 Wh and 348.41 Wh per query when using Llama-3.1-Instruct 8B and 70B as backend LLMs, whereas LATS consumes 22.76 Wh and 158.48 Wh (Table III). By contrast, a conventional single-turn LLM inference (ShareGPT) requires only 0.32 Wh (8B) and 2.55 Wh (70B) per query. These figures correspond to a $62.1\times$ – $136.5\times$ increase in GPU energy per query under agent-based test-time scaling (vs. single-turn LLM inference).

Based on recent estimates, ChatGPT serves roughly 500 million to 1.27 billion weekly active users (WAU) [15], [58], [60], [73], which corresponds to approximately 71.4 million to

TABLE III
ACCURACY, LATENCY, AND GPU ENERGY CONSUMPTION WHEN
SERVICING A SINGLE AGENT REQUEST ON HOTPOTQA. NUMBERS IN
PARENTHESES INDICATE THE RELATIVE INCREASE OVER SHAREGPT (THE
CONVENTIONAL SINGLE-TURN INFERENCE).

	Accuracy (%)	Latency (seconds)	Energy (Wh/query)
8B	ShareGPT	—	4.23 (1×)
	Reflexion	38	649.34 (153.7×)
	LATS	80	380.90 (90.1×)
70B	ShareGPT	—	6.40 (1×)
	Reflexion	67	720.00 (112.6×)
	LATS	82	305.67 (47.8×)

TABLE IV
DATACENTER-WIDE POWER DEMAND UNDER CURRENT AND FUTURE
TRAFFIC SCENARIOS (71.4 MILLION AND 13.7 BILLION QUERIES/DAY),
ASSUMING THE HOTPOTQA BENCHMARK.

	Power @ 71.4 Million Queries/day (Watts)	Power @ 13.7 Billion Queries/day (Watts)
8B	ShareGPT	1.0 M
	Reflexion	123.6 M
	LATS	67.7 M
70B	ShareGPT	7.6 M
	Reflexion	1.0 G
	LATS	471.5 M

181.4 million daily active users (DAU). Assuming the conservative estimate of 71.4 million DAU and that each user submits just a “single” agentic query per day, Reflexion’s daily GPU energy consumption would be approximately 2.97 GWh for the 8B model and 24.89 GWh for the 70B model. Although our analysis does not account for LLM request batching [32], [98], which can amortize execution overheads, the estimate remains conservative for three reasons: (1) it represents a lower-bound based on the conservative DAU estimate of 71.4 million and assumes just one query per user, despite accelerating adoption and increasing user demand, (2) it includes only GPU energy, omitting overheads from CPU, memory, networking, storage, and cooling, and (3) even the larger 70B model considered in our study is orders of magnitude smaller than today’s large-scale LLMs, which now reach hundreds of billions to trillions of parameters [3], [13], [18], [46].

Even under these modest assumptions, the projected demand rivals the daily electricity consumption of Seattle and its surrounding area (24.8 GWh) [68]. As AI agents become increasingly embedded in everyday applications, their query volume could approach, or exceed, that of traditional search engines. For instance, Google Search processes over 13.7 billion queries per day [50], roughly 192× the 71.4 million agentic queries assumed above. If this growth in user base and usage persists, AI infrastructure demand could rise dramatically, potentially exceeding sustainable limits and underscoring the significant challenges posed by test-time scaling.

Datacenter-wide power demands. We now move on to estimating the datacenter-wide power requirements to sustain the aforementioned AI service demands, assuming today’s (ChatGPT’s 71.4 million queries per day, assuming the conser-

vative ChatGPT DAU estimate and one agentic query per user) and tomorrow’s (Google search’s 13.7 billion queries per day) AI traffic. Table IV translates the per-query GPU energy consumption numbers into datacenter-level power requirements, computed by $P = (\text{Wh/query}) \times ((\text{Queries/Day}) / (24 \text{ hours}))$. Under today’s 71.4 million DAU load, single-turn ShareGPT (70B) requires roughly 7.6 MW, well within the tens-of-megawatts envelope typical of modern datacenters [27], [88]. However, assuming similar traffic levels for AI agents, even the lighter 8B-based agents demand 67.7–123.6 MW, comparable to the power draw of a mid-sized U.S. city, while 70B-based agents approach 1 GW, nearly three orders of magnitude higher than the single-turn LLM baseline. Strikingly, this gigawatt-scale power requirement aligns with the announced budget for OpenAI’s multi-gigawatt Stargate facility [55], which is intended to support *future* AI model deployments. Yet, our analysis suggests that such infrastructure may already be necessary to support agentic systems under today’s traffic levels. Overall, our estimates indicate that even modest user traffic (on the order of tens of millions of queries per day) becomes gigawatt-scale once per-query energy exceeds ~100 Wh, a threshold representative of current agentic workloads.

If we were to scale the same per-query figures to Google’s 13.7 billion daily searches, the power numbers would raise single-turn ShareGPT (70B) to 1.5 GW and Reflexion (70B) to nearly 200 GW, far beyond any announced datacenter project (e.g., Meta’s recently announced 5 GW AI datacenter Hyperion is scheduled for deployment in 2030 [74]) and exceeding the power budgets of many national grids. To put this number into perspective, a 200 GW is almost half of the *entire* U.S. grid’s average load (which amounts to $4,178 \times 10^3 \text{ GWh} / (365 \times 24 \text{ hours}) = 476.9 \text{ GW}$ [81]), a scale usually discussed only for nation-wide decarbonization plans, not for a single industry or technology, one that fundamentally reshapes generation, transmission, and sustainability planning.

Sustainability challenges of agentic test-time scaling. Collectively, our findings show that AI agent performance does not scale proportionally with the associated compute, energy, and power costs. Once accuracy saturates, additional test-time scaling yields diminishing returns while imposing substantial system-level burdens. This cost inefficiency is not merely theoretical; it poses concrete constraints on real-world deployments. For instance, OpenAI’s Deep Research [57], designed for complex multi-step reasoning, can take up to 30 minutes per request [56]. To keep infrastructure costs manageable, OpenAI limits usage to 25 runs every 30 days for ChatGPT Plus users [56]. These limits highlight the financial and computational challenges of sustaining AI systems that rely heavily on intensive test-time computation.

Based on these findings, we argue that building scalable and sustainable AI agents requires moving away from unconstrained test-time scaling. Instead, AI agents should be designed with compute-aware agentic workflows that deliver strong performance through efficient inference, rather than single-handedly relying on extended reasoning depth.

VII. DISCUSSION

Future directions for sustainable AI agent serving.

While the primary objective of our work is to raise awareness within the community about the broader system-level implications of deploying agents—particularly the immense infrastructural costs associated with them—we also highlight several promising directions that we believe will be critical for sustainably serving agentic systems. First, conventional *model-level optimizations* such as quantization [19], [38], [89], distillation [24], [26], [49], [77], sparse architectures [17], [37], [99], and adaptive model routing [16], [59] will remain essential for reducing computational and memory demands. For example, constructing multi-agent systems that combine a heterogeneous mix of small and large language models (SLMs and LLMs), and dynamically selecting the appropriate model depending on the agent’s role and task significance (e.g., planning vs. acting), can substantially reduce both operational cost and latency—an approach also advocated by [6]. Second, for AI agents that are not strictly latency-sensitive, *carbon-aware computing* [22], [35], [65] that migrates parts of the execution to compute instances incurring lower carbon intensity or electricity costs can provide both environmental and economic benefits. Finally, *adaptive scaling strategies* [64], [76], [91] that dynamically adjust compute resources based on task difficulty and importance enable agents to allocate GPU resources more efficiently, avoiding over-provisioning while maintaining quality of service. Together, these directions highlight exciting opportunities for improving the efficiency, sustainability, and scalability of serving AI agent workloads.

Agent serving under SLA constraints. The industry has only recently begun exploring agentic systems, so there are currently no well-established or widely accepted SLA standards. Consequently, we did not (and realistically could not) conduct our analysis with respect to a specific SLA target. The goal of this work is to characterize energy–efficiency trade-offs across diverse agent configurations, rather than to optimize for a fixed latency constraint. Considering the current direction of agentic system development, agents are generally allowed to spend additional computation time to achieve higher reasoning quality. Our analysis intentionally examines this behavior to highlight the inefficiency of unrestricted test-time scaling and to motivate more energy-aware SLA design in future agent deployments. A detailed exploration of efficient agent serving under SLA constraints is left as future work.

VIII. RELATED WORK

AI agent workflows. Recent advances in LLM-based AI agents have introduced diverse workflows that combine language-based reasoning with external tool use. Single-agent frameworks (e.g., ReAct [96], Reflexion [72], LATS [102], LLMCompiler [31]) enhance decision-making through iterative reasoning, tool execution, and reflection. Multi-agent systems, such as CAMEL [36] and AutoGen [87], further extend these capabilities by structuring task execution, communication, and coordinated behaviors among multiple agents. While these workflows substantially improve their capabilities and

behavioral flexibility, their system-level implications remain underexplored. This work provides the first comprehensive analysis of representative AI agents, offering insights into the efficiency and scalability of agentic systems.

AI agent interfaces for tool-augmented reasoning. In parallel with behavioral advancements in AI agents, recent efforts have focused on standardizing AI agent APIs and protocols to facilitate broader integration and deployment. OpenAI’s function-calling interface [54] defines a structured mechanism for API invocation, enabling agents to interact with tools in a verifiable and consistent manner. Anthropic’s Model-Context-Protocol (MCP) [5] further formalizes how agents manage context and interact with tools. Google’s Agent-to-Agent (A2A) protocol [21] complements these efforts by specifying a standard for multi-agent communication. Although these contributions primarily standardize the interfaces and protocols for agent interaction, our work takes an orthogonal system-level perspective, uncovering the AI infrastructural challenges posed by agentic workloads under test-time scaling.

System-level optimization of AI agents. LLMCompiler [31], Alto [67], and Ayo [78] reduce inference latency by enabling pipelined and parallel execution across reasoning steps. Autellix [43] optimizes latency through queue-aware scheduling, while AI Metropolis [90] and Murakkab [9] improve multi-agent coordination and resource isolation. While these works focus on optimizing specific components such as scheduling or execution flow, our study provides a broader characterization of infrastructural behaviors and efficiency trade-offs across diverse AI agents at scale.

LLM inference optimization techniques. The AI community has only recently begun exploring agents, making it both unclear and highly challenging to determine the most effective methodology for applying various LLM-focused optimization techniques to agentic systems. To maintain clarity and generality in our analysis, this paper focuses on fundamental and widely adopted LLM inference optimizations that are readily available in existing AI serving frameworks [32], [100]. A comprehensive exploration of all the latest LLM optimizations in the literature for agentic systems is beyond the scope of this paper, so we provide a summary of recent LLM inference optimizations and discuss their applicability to agents below.

In terms of KV cache management, *hierarchical caching* [20], [29], [33] and *non-prefix KV cache reuse* [93] approaches extend the naive prefix caching, enabling more efficient KV cache reuse. *Token pruning* [1], [23], [42] or *KV cache compression* [40], [41], or model architectural improvement like grouped-query attention [4] and multi-head attention [39] reduces the memory footprint of the KV cache, which will be especially helpful for agent workloads with long contexts. Regarding decoding, *Speculative decoding* [34] predicts multiple candidate tokens and validates in parallel to reduce decoding latency. In agents, speculative decoding can potentially become effective as their reasoning often generates predictable schema patterns (e.g., JSON structures or function arguments), which will increase the acceptance rate of speculative branches and improve overall decoding

throughput. *Prefill-decode disaggregation* [52], [61], [101] allows flexible and efficient resource allocation by decoupling the compute-intensive prefill phase from the memory-bound decode phase. For agents with long-context that incur substantial prefill computation load, disaggregation mitigates interference between prefill and decoding workloads, leading to more stable performance and improved overall efficiency.

IX. CONCLUSION

This paper provides the first system-level characterization of AI agents from an AI infrastructure perspective. While these LLM-based agents demonstrate powerful reasoning capabilities, they also introduce substantial energy overheads that are orders of magnitude higher than conventional single-turn LLM inference. Our analysis shows that common agent design patterns incur heavy latency penalties and infrastructure costs, especially when deployed at scale. Moreover, test-time scaling yields sharply diminishing returns in accuracy, challenging the cost-effectiveness of current agent implementations.

These findings underscore an urgent need to rethink agent architecture and workflow design. Rather than relying on brute-force test-time scaling, future agents should adopt compute-aware reasoning strategies that optimize accuracy per unit cost. This includes smarter scheduling, caching, prompt engineering, and hybrid scaling approaches that adapt to deployment constraints. By exposing the hidden costs of agentic reasoning and offering actionable insights into their infrastructure impact, we hope this work informs future system and algorithm co-design for scalable and sustainable AI agents.

ACKNOWLEDGMENT

This work was partly supported by Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00438851, (SW Starlab) High-performance Privacy-preserving Machine Learning System and System Software), (No.RS-2024-00395134, DPU-Centric Datacenter Architecture for Next-Generation AI Devices), (No.RS-2025-02264029, Implementation and Validation of an AI Semiconductor-Based Data Center Composable Cluster Infrastructure, 30%), and by Samsung Research Funding Center of Samsung Electronics (SRFC-IT2402-03). Minsoo Rhu is the corresponding author.

REFERENCES

- [1] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," in *Proceedings of Machine Learning and Systems*, 2024.
- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [3] M. AI, "Kimi k2: Open agentic intelligence," <https://moonshotai.github.io/Kimi-K2/>, 2025.
- [4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [5] Anthropic, "Model Context Protocol (MCP)," 2024. [Online]. Available: <https://docs.anthropic.com/en/docs/agents-and-tools/mcp/>
- [6] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic ai," *arXiv preprint arXiv:2506.02153*, 2025.
- [7] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [8] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24, 2024.
- [9] G. I. Chaudhry, E. Choukse, Íñigo Goiri, R. Fonseca, A. Belay, and R. Bianchini, "Towards Resource-Efficient Compound AI Systems," in *arxiv.org*, 2025.
- [10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," in *arxiv.org*, 2021.
- [11] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *International conference on computers and games*. Springer, 2006.
- [12] A. de Vries, "The growing energy footprint of artificial intelligence," *Joule*, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2542435123003653>
- [13] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [14] Dell'Oro Group, "Data Center Capex to Surpass \$1 Trillion by 2029, According to Dell'Oro Group," 2025. [Online]. Available: <https://www.delloro.com/news/data-center-capex-to-surpass-1-trillion-by-2029/>
- [15] DemandSage, "Chatgpt statistics and facts (2024-2025)," 2025. [Online]. Available: <https://www.demandsage.com/chatgpt-statistics/>
- [16] D. Ding, A. Mallick, S. Zhang, C. Wang, D. Madrigal, M. D. C. H. Garcia, M. Xia, L. V. Lakshmanan, Q. Wu, and V. Rühle, "Best-route: Adaptive llm routing with test-time optimal compute," *arXiv preprint arXiv:2506.22716*, 2025.

- [17] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [18] Fello AI, "Gemini 1.5 pro: All you need to know about this near-perfect ai model," 2024. [Online]. Available: <https://felloai.com/2024/09/google-gemini-pro-1-5-all-you-need-to-know-about-this-near-perfect-ai-model/>
- [19] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [20] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-Efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 111–126. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [21] Google, "Announcing the Agent2Agent Protocol (A2A)," 2025. [Online]. Available: <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- [22] V. U. Gsteiger, P. H. Long, Y. Sun, P. Javanrood, and M. Shahrad, "Caribou: Fine-grained geospatial shifting of serverless applications for sustainability," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 403–420.
- [23] X. Gu, T. Pang, C. Du, Q. Liu, F. Zhang, C. Du, Y. Wang, and M. Lin, "When attention sink emerges in language models: An empirical view," *arXiv preprint arXiv:2410.10781*, 2024.
- [24] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.
- [25] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring Mathematical Problem Solving With the MATH Dataset," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [27] IBM, "What is a hyperscale data center?" 2024. [Online]. Available: <https://www.ibm.com/think/topics/hyperscale-data-center>
- [28] Industry Week, "The Success of US Chip Manufacturing Hinges on Our Electric Grid," 2024. [Online]. Available: <https://www.industryweek.com/technology-and-iiot/energy/article/21284413/the-success-of-us-chip-manufacturing-hinges-on-our-electric-grid>
- [29] J. Jeong and J. Ahn, "Accelerating Llm serving for multi-turn dialogues with efficient resource management," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025.
- [30] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. Mahoney, K. Keutzer, and A. Gholami, "LLMCompiler: An LLM Compiler for Parallel Function Calling," 2023. [Online]. Available: <https://github.com/SqueezeAILab/LLMCompiler>
- [31] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM Compiler for Parallel Function Calling," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [32] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the ACM Symposium on Operating System Principles (SOSP)*, 2023.
- [33] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- [34] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [35] B. Li, S. Samsi, V. Gadepally, and D. Tiwari, "Clover: Toward sustainable ai with carbon-aware machine learning inference service," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.
- [36] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [37] W. Liang, L. Yu, L. Luo, S. Iyer, N. Dong, C. Zhou, G. Ghosh, M. Lewis, W.-t. Yih, L. Zettlemoyer *et al.*, "Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models," *arXiv preprint arXiv:2411.04996*, 2024.
- [38] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [39] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.
- [40] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," in *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2024.
- [41] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24, 2024.
- [42] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Ré, and B. Chen, "Deja vu: contextual sparsity for efficient llms at inference time," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23, 2023.
- [43] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, "Autellix: An efficient serving engine for llm agents as general programs," 2025.
- [44] Meta, "Llama-3.1-70B-Instruct," 2025. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.1-70B-Instruct>
- [45] —, "Llama-3.1-8B-Instruct," 2025. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
- [46] Meta AI, "Llama 4: Advancing multimodal intelligence at scale," <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025.
- [47] MLPerf, "MLPerf Inference: Datacenter," [Online]. Available: <https://mlcommons.org/benchmarks/inference-datacenter/>
- [48] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple test-time scaling," 2025. [Online]. Available: <https://arxiv.org/abs/2501.19393>
- [49] S. Muralidharan, S. Turuvekere Sreenivas, R. Joshi, M. Chochowski, M. Patwary, M. Shoeybi, B. Catanzaro, J. Kautz, and P. Molchanov, "Compact language models via pruning and knowledge distillation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 41 076–41 102, 2024.
- [50] Naveen Kumar, "How Many Google Searches Per Day [2025 Data]," 2025. [Online]. Available: <https://www.demandsage.com/google-search-statistics/>
- [51] NVIDIA, "NVIDIA DCGM Documentation," [Online]. Available: <https://docs.nvidia.com/datacenter/dcgmlatest/index.html>
- [52] —, "NVIDIA Dynamo Platform," [Online]. Available: <https://developer.nvidia.com/dynamo>
- [53] OpenAI, "Introducing ChatGPT," 2022. [Online]. Available: <https://openai.com/index/chatgpt/>
- [54] OpenAI, "OpenAI Function Calling," 2023. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling?api-mode=chat>
- [55] —, "Announcing The Stargate Project," 2025. [Online]. Available: <https://openai.com/index/announcing-the-stargate-project/>
- [56] —, "Deep Research FAQ," 2025. [Online]. Available: <https://help.openai.com/en/articles/10500283>
- [57] —, "Introducing Deep Research," 2025. [Online]. Available: <https://openai.com/index/introducing-deep-research/>
- [58] OpenAI, "New funding to build towards AGI," 2025. [Online]. Available: <https://openai.com/index/march-funding-updates/>
- [59] P. Panda, R. Magazine, C. Devaguptapu, S. Takemori, and V. Sharma, "Adaptive llm routing under budget constraints," *arXiv preprint arXiv:2508.21141*, 2025.
- [60] M. Paris, "Chatgpt hits 1 billion users, openai ceo says, doubled in weeks," April 2025. [Online]. Available: <https://www.forbes.com/sites/martineparis/2025/04/12/chatgpt-hits-1-billion-users-openai-ceo-says-doubled-in-weeks/>
- [61] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase

- splitting,” in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2024.
- [62] Pranshu Verma and Shelly Tan, “A bottle of water per email: the hidden environmental costs of using AI chatbots,” 2024. [Online]. Available: <https://www.washingtonpost.com/technology/2024/09/18/energy-ai-use-electricity-water-data-centers/>
- [63] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” in *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [64] Y. Qu, M. Y. Yang, A. Setlur, L. Tunstall, E. E. Beeching, R. Salakhutdinov, and A. Kumar, “Optimizing test-time compute via meta reinforcement fine-tuning,” *arXiv preprint arXiv:2503.07572*, 2025.
- [65] A. Radovanović, R. Konigstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care *et al.*, “Carbon-aware computing for datacenters,” *IEEE Transactions on Power Systems*, vol. 38, no. 2, pp. 1270–1280, 2022.
- [66] Reuters, “Meta to invest \$10 billion for Louisiana data center,” 2024. [Online]. Available: <https://www.reuters.com/technology/meta-invest-10-billion-louisiana-data-center-2024-12-04/>
- [67] K. Santhanam, D. Raghavan, M. S. Rahman, T. Venkatesh, N. Kunjal, P. Thaker, P. Levis, and M. Zaharia, “ALTO: An Efficient Network Orchestrator for Compound AI Systems,” in *Proceedings of the 4th Workshop on Machine Learning and Systems*, 2024.
- [68] Seattle City Light, “Fingertip Facts,” 2024. [Online]. Available: <https://www.seattle.gov/documents/Departments/CityLight/FingertipFacts.pdf>
- [69] SemiAnalysis, “The Inference Cost Of Search Disruption – Large Language Model Cost Analysis,” 2023. [Online]. Available: <https://semianalysis.com/2023/02/09/the-inference-cost-of-search-disruption/>
- [70] ShareGPT Team, “Sharegpt,” 2023. [Online]. Available: <https://sharegpt.com>
- [71] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” 2023. [Online]. Available: <https://github.com/noahshinn/reflexion>
- [72] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [73] Similarweb, “Chatgpt.com website performance - june 2025,” 2025. [Online]. Available: https://pro.similarweb.com/#/digitalsuite/websiteseanalysis/overview/website-performance/*999/1m?webSource=Total&key=chatgpt.com
- [74] J. Singh and A. Soni, “Meta’s Zuckerberg pledges hundreds of billions for AI data centers in superintelligence push,” 2025. [Online]. Available: <https://www.reuters.com/business/zuckerberg-says-meta-will-invest-hundreds-billions-superintelligence-2025-07-14/>
- [75] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling llm test-time compute optimally can be more effective than scaling model parameters,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.03314>
- [76] —, “Scaling llm test-time compute optimally can be more effective than scaling model parameters,” *arXiv preprint arXiv:2408.03314*, 2024.
- [77] S. T. Sreenivas, S. Muralidharan, R. Joshi, M. Chochowski, A. S. Mahabaleshwarkar, G. Shen, J. Zeng, Z. Chen, Y. Suhara, S. Diao *et al.*, “Llm pruning and distillation in practice: The minitron approach,” *arXiv preprint arXiv:2408.11796*, 2024.
- [78] X. Tan, Y. Jiang, Y. Yang, and H. Xu, “Towards end-to-end optimization of llm-based applications with ayo,” in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [79] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” in *arxiv.org*, 2023.
- [80] B. Tran and P. Attorney, “Semiconductor Manufacturing Energy Consumption: How Green Is the Chip Industry?” 2025. [Online]. Available: <https://patentpc.com/blog/semiconductor-manufacturing-energy-consumption-how-green-is-the-chip-industry-latest-stats>
- [81] U.S. Energy Information Administration (EIA), “Electricity explained - Electricity generation, capacity, and sales in the United States,” 2024. [Online]. Available: <https://www.eia.gov/energyexplained/electricity/electricity-in-the-us-generation-capacity-and-sales.php>
- [82] vLLM, “vLLM Documentation.” [Online]. Available: <https://docs.vllm.ai/en/stable/>
- [83] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [84] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [85] Wikipedia contributors, “Wikipedia api,” 2025. [Online]. Available: https://www.mediawiki.org/wiki/API:Main_page
- [86] Wolfram Alpha LLC, “Wolfram alpha apis,” 2025. [Online]. Available: <https://products.wolframalpha.com/api/>
- [87] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “Autogen: Enabling next-gen LLM applications via multi-agent conversations,” in *First Conference on Language Modeling (CoLM)*, 2024.
- [88] xAI Colossus, “Colossus — xAI,” 2025. [Online]. Available: <https://x.ai/colossus>
- [89] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.
- [90] Z. Xie, H. Kang, Y. Sheng, T. Krishna, K. Fatahalian, and C. Kozyrakis, “AI Metropolis: Scaling Large Language Model-Based Multi-Agent Simulation with Out-of-Order Execution,” in *arxiv.org*, 2024.
- [91] W. Yang, S. Ma, Y. Lin, and F. Wei, “Towards thinking-optimal scaling of test-time compute for llm reasoning,” *arXiv preprint arXiv:2502.18080*, 2025.
- [92] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [93] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “Cacheblend: Fast large language model serving for rag with cached knowledge fusion,” in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys ’25, 2025.
- [94] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents,” in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- [95] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [96] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [97] —, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2023. [Online]. Available: <https://github.com/ysymyth/ReAct>
- [98] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [99] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao *et al.*, “Native sparse attention: Hardware-aligned and natively trainable sparse attention,” *arXiv preprint arXiv:2502.11089*, 2025.
- [100] L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, “Sglang: Efficient execution of structured language model programs,” *Advances in neural information processing systems*, vol. 37, pp. 62 557–62 583, 2024.

- [101] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [102] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [103] —, “Official Repo of Language Agent Tree Search (LATS),” 2024. [Online]. Available: <https://github.com/lapisrocks/LanguageAgentTreeSearch>
- [104] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.